

SISTEMAS OPERATIVOS

Conferencia 1 · Tema I: Introducción

Carrera: Ingeniería Informática

Año: 3ro **Período:** 2do **Tipo de curso:** CRD

Disciplina: Infraestructura de Sistemas Informáticos

Profesora: Ing. Claudia Vecino Fernández

Correo: claudiavf@uho.edu.cu

Teléfono: 54196249

Objetivo: Analizar el concepto de Sistema Operativo en toda su dimensión, a partir de sus características y su evolución histórica.

Sumario

- Sistemas de operación y funciones principales.
- Evolución histórica.
 - Primera etapa.
 - Monitores simples.
 - Multiprogramación.
 - Tiempo compartido.
 - Tiempo real.
- Generaciones de los sistemas operativos.
- Sistemas multiprocesadores.
- Protección.
- Procesos y tabla de procesos.
- Servicios del sistema operativo.
- Programas del sistema.
- Llamadas al sistema.
- Intérprete de comandos ("Shell").
- Estructura de un sistema operativo.

Distribución del Contenido y Evaluación

Distribución del Contenido	Sistema de Evaluación
• 8 Conferencias • 8 Clases Prácticas • 2 Laboratorios	• 2 Seminarios • 2 Pruebas Parciales • 1 Prueba Final

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos. Págs. 1–27.
- Tanenbaum, A. S. Operating System: Design and Implementation. Págs. 1–42.
- Peterson, J. y Silberschatz, A. Operating System Concepts. Págs. 1–35, 39–54.

1. Sistemas de Operación y Funciones Principales

Todo sistema de cómputo puede dividirse, en términos generales, en dos grandes partes: hardware (aseguramiento técnico) y software (aseguramiento de programas).

El software hace útil al hardware y puede clasificarse en dos grandes grupos: los programas del sistema (software de base), que gestionan la operación de la computadora, y el software de aplicación, que realiza tareas útiles para los usuarios.

Entre los programas del sistema se encuentran: sistemas operativos (los más importantes), compiladores, intérpretes, editores, entre otros. El sistema operativo es la primera capa de software que se coloca sobre el hardware, separando a los usuarios de la complejidad de este.

Definición: Un sistema operativo es una interfaz entre los usuarios y el hardware que brinda un ambiente donde los programas se ejecutan con facilidad y conveniencia, logrando al mismo tiempo un uso eficiente de los recursos disponibles.

Todo sistema operativo cumple dos funciones principales:

Función	Descripción
Máquina ampliada / virtual	Brinda facilidades a los usuarios, ocultando las complejidades del hardware. Ejemplo: el usuario trabaja con archivos en lugar de sectores de disco.
Administrador de recursos	Asigna de forma ordenada y controlada los recursos del sistema (CPU, memoria, dispositivos) entre los programas que compiten por ellos, buscando la eficiencia.

La primera función antes indicada es la razón primaria de la existencia de los sistemas operativos y esto resulta más evidente cuando se trata de las computadoras de uso personal.

La meta de la eficiencia es importante para sistemas de cómputo grandes y sistemas compartidos de múltiples usuarios ("Shared Multi-Users Systems"). En este caso, no se quiere decir que la vista de máquina ampliada no siga constituyendo una función fundamental.

En el pasado, las consideraciones de eficiencia fueron frecuentemente más importantes que las de facilidades de uso y por ello la teoría de los sistemas operativos se concentró en el uso óptimo de los recursos. Hoy, con la gran difusión de computadoras personales existe un mayor énfasis en la satisfacción del usuario y a ello se debe el desarrollo de los ambientes GUI ("Graphic User Interface").

Aun cuando no existe una definición absoluta de que es un sistema operativo y resulta más claro llegar a su entendimiento por la vía de comprender que hacen, ensayaremos una muy simple.

Un sistema operativo es una interface entre los usuarios y el hardware que brinda a estos un ambiente donde se ejecutan con facilidad y conveniencia sus programas y al mismo tiempo se logra un uso eficiente de los recursos disponibles.

Los sistemas operativos se diferencian de los demás programas del sistema en que corren en modo kernel (también llamado supervisor, protegido o sistema), mientras que los programas de usuario corren en modo usuario. Esta dualidad es la base del mecanismo de protección del sistema.

2. Evolución Histórica

La evolución de los sistemas operativos no puede analizarse de forma aislada: siempre estuvo ligada al avance del hardware. Los SO se crearon para ampliar las posibilidades del hardware, y a su vez pusieron en evidencia la necesidad de nuevas capacidades en los equipos.

2.1 Primera Etapa

Se caracteriza por la ausencia total de sistema operativo. Solo existía el hardware y el programador era a la vez el operador. El uso de las máquinas se organizaba por reservación de tiempo.

Con el paso del tiempo aparecieron: lectores de tarjetas, impresoras de línea, bandas magnéticas, ensambladores, cargadores, enlazadores y bibliotecas con funciones comunes (incluyendo manejadores de dispositivos o device drivers). Estas últimas pueden considerarse embriones de los futuros sistemas operativos.

2.2 Monitores Simples

La aparición de los compiladores complicó la operación y condujo al procesamiento por lotes (batch) con operadores profesionales. Para reducir el tiempo perdido entre trabajos surgieron los monitores residentes: pequeños programas que permanecían en memoria y transferían automáticamente el control de un programa a otro, guiados por tarjetas de control perforadas.

Los monitores residentes constaban de cuatro componentes principales: vectores de interrupción y trampas, manejadores de dispositivos, cargador e intérprete de tarjetas de control.

Las diferencias de velocidad entre CPU y dispositivos de E/S llevaron a soluciones como el uso de buffers y, posteriormente, el SPOOLING (Simultaneous Peripheral Operation On-Line), basado en discos magnéticos que permitían lecturas y escrituras simultáneas en distintas zonas.

SPOOLING: Técnica que permite solapar la entrada, salida y cálculo de trabajos diferentes, utilizando el disco como almacenamiento intermedio (job pool). Esto habilitó la planificación de trabajos (Job Scheduling).

2.3 Multiprogramación

La multiprogramación es el concepto central de los sistemas operativos modernos. Consiste en mantener varios trabajos en memoria simultáneamente: cuando el trabajo activo debe esperar (por ejemplo, por una operación de E/S), el CPU se asigna a otro trabajo listo para ejecutar.

Este mecanismo aumenta significativamente la utilización del CPU, y su implementación requirió el desarrollo de técnicas de administración de memoria (Memory Management).

2.4 Tiempo Compartido

La multiprogramación hizo posible que múltiples usuarios interactuaran simultáneamente con el sistema desde terminales individuales, compartiendo el CPU. A cada usuario se le asigna el procesador por un breve período (quantum) de forma rotatoria, dando la impresión de tener una máquina dedicada.

2.5 Tiempo Real

Los sistemas de tiempo real también usan multiprogramación, pero con restricciones estrictas de tiempo en el procesamiento. Se emplean para controlar procesos físicos como experimentos científicos, equipos industriales o médicos.

3. Generaciones de los Sistemas Operativos

Algunos autores, como Tanenbaum, analizan la evolución histórica de los sistemas operativos asociándola a las generaciones del hardware:

Generación	Período / Tecnología	Características del SO
1ª	1945–1955 Válvulas de vacío	Sin sistema operativo. Programación manual con tableros de conexiones.
2ª	1955–1965 Transistores	Monitores simples y procesamiento por lotes. Ejemplos: FMS, IBSYS.

Sistemas Operativos Conferencia 1		Ingeniería Informática · 3er Año
3ª	1965–1980 Circuitos integrados	SPOOLING, multiprogramación y tiempo compartido. Ejemplos: OS/360, MULTICS, primeras versiones de UNIX.
4ª	1980–2000 VLSI / PCs	Computadoras personales, GUI, MS-DOS, UNIX. Aparece Linux (Linus Torvalds, 1991).
5ª	2000–hoy Multicore / Cloud / Móvil	Sistemas distribuidos, virtualización, contenedores (Docker), Android, iOS. Linux domina servidores y sistemas embebidos.

 **Nota actualizada:** Linux fue iniciado por Linus Torvalds en 1991 y hoy domina servidores, dispositivos embebidos (Android) y supercómputo. Es de distribución libre y de código abierto.

4. Sistemas Multiprocesadores

El término multiprocesamiento hace referencia a la existencia en un sistema de cómputo de dos o más CPU que comparten memoria y periféricos. Sus ventajas principales son el incremento de la potencia de cómputo y la posibilidad de paralelismo real.

Los sistemas operativos que gestionan estas configuraciones reciben el nombre de sistemas operativos multiprocesador. Su diseño es similar al de los uniprocadores, pero debe considerar el acceso simultáneo de múltiples procesadores a recursos compartidos (secciones críticas, coherencia de caché, etc.).

 **Nota actualizada:** En la actualidad, prácticamente todos los procesadores modernos son multinúcleo (multicore), por lo que la programación concurrente y paralela es una habilidad esencial. Tecnologías como OpenMP, MPI y los hilos POSIX (pthreads) son herramientas clave en este contexto.

5. Protección

En los primeros sistemas solo se ejecutaba un programa a la vez, por lo que la protección no era necesaria. Al incorporarse la multiprogramación, varios programas comparten memoria y recursos, y un fallo en uno puede afectar al resto o al propio sistema operativo.

Los recursos que obligatoriamente deben protegerse son: la entrada/salida, la memoria y la CPU.

5.1 Protección de Entrada/Salida

Se logra declarando como privilegiadas las instrucciones de E/S, de modo que solo el sistema operativo puede ejecutarlas. Los programas de usuario deben solicitar la E/S a través de las llamadas al sistema, que validan la solicitud antes de ejecutarla.

5.2 Modo Kernel y Modo Usuario

El hardware provee un bit de modo que indica si el código en ejecución es el SO (modo kernel/privilegiado) o un programa de usuario (modo usuario). Solo en modo kernel se pueden ejecutar instrucciones privilegiadas y modificar estructuras críticas del sistema.

5.3 Protección de Memoria

En monoprogramación basta un registro límite que impida al programa de usuario acceder al área del SO. En ambientes multiprogramados se emplean dos registros (base y límite) que delimitan el área de memoria válida para el programa en ejecución actual.

Al cambiar de proceso, el SO actualiza estos registros con los valores correspondientes al nuevo programa. Las instrucciones para modificar estos registros son privilegiadas.

Esquema moderno: Los procesadores actuales utilizan unidades de gestión de memoria (MMU) con paginación y segmentación, que ofrecen protección y aislamiento mucho más sofisticados que los simples registros base/límite.

5.4 Protección del Procesador

Para evitar que un programa de usuario monopolice el CPU indefinidamente mediante un bucle infinito, el hardware incorpora un temporizador (timer). El SO carga un contador en el timer; al llegar a cero, se genera una interrupción que devuelve el control al SO. Las instrucciones que modifican el timer son privilegiadas.

6. Procesos y Tabla de Procesos

Definición: Un proceso es un programa en ejecución. A diferencia del programa (código estático), el proceso incluye el código, los datos, la pila, el contador de programa, los registros y todo lo que refleja el estado de ejecución en un instante determinado.

El concepto de proceso es fundamental en la multiprogramación: permite guardar y restaurar el estado de ejecución cuando el CPU es retirado temporalmente a un programa para asignárselo a otro.

Para gestionar los procesos activos el SO mantiene la tabla de procesos: un arreglo de estructuras, cada una con los datos de estado de un proceso. Campos típicos incluyen: estado del proceso, valores de registros, contador de programa, identificador (PID), área de memoria, tabla de archivos abiertos, directorio actual, entre otros.

Todo proceso se identifica por un entero único denominado PID (Process Identifier). Un proceso puede crear procesos hijos, formando estructuras arbóreas de procesos.

7. Servicios del Sistema Operativo

El SO brinda servicios a dos niveles: a los usuarios, mediante los Programas del Sistema, y a los procesos, mediante las Llamadas al Sistema.

7.1 Programas del Sistema

Son una colección de programas suministrados con el SO que permiten realizar operaciones comunes: manipulación de archivos (crear, eliminar, renombrar, imprimir), obtención de información de estado (fecha, hora, espacio en disco), editores de texto, compiladores, etc.

En sistemas con interfaz gráfica (GUI), los programas del sistema son los que se ejecutan al seleccionar opciones de menú o activar iconos.

7.2 Llamadas al Sistema

Constituyen la interfaz entre los procesos y el SO. Permiten que un programa solicite servicios del kernel de forma controlada. En Linux, por ejemplo, se accede a ellas mediante la instrucción `syscall`. En C, las funciones de la biblioteca estándar (`open`, `read`, `write`, `fork`, etc.) encapsulan estas llamadas.

Categoría	Ejemplos de operaciones
Control de procesos	<code>fork()</code> , <code>exec()</code> , <code>exit()</code> , <code>wait()</code> , <code>kill()</code> — crear, ejecutar, terminar, esperar procesos.
Manipulación de archivos	<code>open()</code> , <code>close()</code> , <code>read()</code> , <code>write()</code> , <code>rename()</code> , <code>unlink()</code> — gestión del sistema de archivos.
Manipulación de dispositivos	<code>ioctl()</code> , <code>read()</code> , <code>write()</code> — solicitar y liberar dispositivos de E/S.
Intercambio de información	<code>gettimeofday()</code> , <code>stat()</code> , <code>getpid()</code> — consultar fecha, atributos, identidad.
Comunicación entre procesos	<code>pipe()</code> , <code>socket()</code> , <code>shmget()</code> — IPC: tuberías, sockets, memoria compartida.

8. intérprete de Comandos (Shell)

El intérprete de comandos es uno de los programas del sistema más importantes. Sirve de interfaz entre el usuario y el sistema operativo: toma el control una vez que el SO ha sido cargado o cuando un usuario inicia sesión en un sistema de tiempo compartido.

Ha recibido distintos nombres: intérprete de tarjetas de control, procesador de comandos de consola, Shell (UNIX/Linux). Su función básica consiste en leer el comando, validarlo y ejecutarlo (ya sea directamente o lanzando el programa correspondiente como proceso hijo).

 **Nota actualizada:** En sistemas Unix/Linux actuales los shells más utilizados son Bash, Zsh y Fish. En Windows, PowerShell ha reemplazado al cmd.exe como intérprete estándar. Los scripts de shell son una herramienta fundamental para la automatización.

9. Estructura de un Sistema Operativo

9.1 Vista Funcional

Los sistemas operativos son programas manejados por eventos (interrupciones o trampas). Si no hay programa que ejecutar, dispositivo que atender o usuario que responder, el SO permanece inactivo.

Los tipos de interrupción son:

- Llamadas al sistema: solicitud de terminación normal o anormal, solicitud de recursos, solicitud de E/S, solicitud de información de estado.
- Interrupción de dispositivo: señal del hardware cuando completa una operación de E/S.
- Error de programa detectado por hardware: división entre cero, acceso a memoria no permitida, instrucción inválida, etc.

9.2 Vista Organizativa: Tipos de Estructura

1. Sistemas Monolíticos

No tienen estructura formal. Son una colección de procedimientos donde cualquiera puede llamar a cualquier otro. Aunque simples, son difíciles de mantener y depurar. Ejemplo histórico: MS-DOS.

2. Sistemas de Capas

El SO se organiza como una jerarquía de capas, cada una construida sobre la anterior. Ejemplo pionero: sistema THE (Dijkstra, 1968). MULTICS generalizó este concepto con anillos concéntricos de protección.

3. Máquinas Virtuales

El núcleo crea múltiples máquinas virtuales que son copias exactas del hardware real, permitiendo ejecutar distintos SO simultáneamente. Hoy esta idea es la base de hipervisores como VMware, VirtualBox, KVM y la tecnología de contenedores (Docker).

4. Modelo Cliente-Servidor

Tendencia moderna: se construye un kernel mínimo (microkernel) y la mayoría de los servicios se implementan como procesos de usuario (servidores). El kernel solo gestiona la comunicación entre clientes y servidores. Ventajas: mayor tolerancia a fallos y facilidad de distribución. Ejemplos: Mach, QNX, MINIX 3.

10. Estudio independiente

1. ¿Cuáles son las dos funciones principales de un sistema operativo? Ejemplifique cada una.
2. ¿Qué diferencia existe entre un proceso y un programa? ¿Por qué esta distinción es esencial para la multiprogramación?
3. Explique el mecanismo de modo kernel y modo usuario. ¿Por qué es necesaria esta distinción para la protección del sistema?

4. ¿En qué consiste la técnica de SPOOLING y qué ventaja introduce respecto al uso de buffers?
5. Compare los cuatro tipos de estructura de un SO (monolítico, capas, máquina virtual, cliente-servidor) señalando ventajas y desventajas de cada uno.
6. Investigue y liste cinco llamadas al sistema disponibles en Linux con su función.

— Fin de la Conferencia 1 —